

Mandate

트레이더가 아니라 조건을 보고 돈을 맡기는 시장

에이전트는 맡긴 돈으로 거래만 할 수 있고, 꺼내 갈 수는 없습니다.
지키기로 한 조건은 운영자의 약속이 아니라 컨트랙트가 지킵니다.

maxLeverageX100

maxDrawdownBps

maxMarkAgeSeconds

allowedMarkets

performanceFeeBps

팀원별 담당 파트

@jiwon000

- MandateRegistry: 에이전트 카탈로그와 조건 대조
- DP Reporter: 차등 프라이버시 성과 공개
- Batch 정산과 Privacy 화면 연동
- Foundry invariant·퍼징 테스트
- 보안 리뷰 4회: registerAgent 수정, 마켓 기능 점검

@yahamang

- MandateVault, MandateRiskGuard: 수탁, 한도 검사, 동결과 청산
- MandateFactory와 Launch 화면: 누구나 등록
- 거래 조건 6종과 수수료 징수
- Perpl 거래소 어댑터: 포크 테스트, 테스트넷 배포
- 테스트넷 데모 서버, 제출 문서, 데모 영상

AI 에이전트에게 돈을 맡기면 생기는 일

상황 A가 트레이딩 에이전트에게 10,000 USDC를 맡깁니다. 약속한 조건은 “레버리지 최대 3배, 고점 대비 손실 3%를 넘으면 중단”.



거래하는 키가 곧 꺼내는 키

보통 에이전트가 쓰는 지갑에 돈이 들어갑니다. 키가 털리거나 운영자가 마음을 바꾸면 돈이 나갑니다.



한도는 약속일 뿐

“3%에서 멈춘다”는 운영자의 말입니다. 레버리지를 10배로 올려도 막을 장치가 없습니다.



운영자가 사라지면 끝

손실 중인 포지션을 닫아 줄 사람이 없습니다. A는 기다리는 것밖에 할 수 없습니다.

질문을 바꿉니다

~~"이 트레이더를 믿을 수 있나?"~~

"이 조건을 받아들일 수 있나?"

조건은 첫 입금 전에 체인에 잠기고, 잠긴 뒤에는 운영자도 바꿀 수 없습니다. 배분자는 성과 주장 대신 잠긴 조건을 읽고 결정합니다.

플랫폼이 정하는 것

조건 종류와 허용 범위 (예: 손실 한도 최대 50%, 마크 유효시간 최대 60초, 성과 수수료 최대 30%)

운영자가 정하는 것

그 범위 안의 실제 값

● 에이전트 운영자

조건을 걸고 스스로 등록합니다. 맡은 돈으로 주문만 낼 수 있습니다.

● 배분자

조건을 읽고 입금합니다. 맡긴 돈은 배분자만 꺼낼 수 있습니다.

● 누구나

조건 위반을 증명해 동결시키고, 청산을 실행하고 보상을 받습니다.

한 번의 사이클: 등록부터 출금까지

1

등록

`createMandate()`

트랜잭션 하나로 볼트 배포, 조건 잠금, 레지스트리 등록. 승인 절차 없음.

2

입금

`allocate()`

돈은 에이전트가 아니라 볼트 컨트랙트로. 배분자는 지분을 받음.

3

주문

`execute()`

모든 주문이 RiskGuard를 먼저 통과해야 거래소에 닿음. 넘으면 트랜잭션째 취소.

4

동결

`poke()`

고점 대비 손실이나 보유 시간이 한도를 넘으면 누구나 증명해 동결. 호출자 보상 0.05%.

5

청산

`unwind()`

누구나 다섯 번에 걸쳐 20%씩 포지션을 닫음. 호출당 보상 0.01%.

6

출금

`withdraw()`

배분자는 최신 가격 기준 지분만큼, 볼트 현금 한도 안에서 출금. 운영자 허락 필요 없음.

1~3은 정상 흐름, 4~5는 위반 뒤 흐름입니다. 6은 동결 뒤에도 열려 있습니다.

핵심 아키텍처

에이전트 키로는 배분자의 돈을 꺼낼 수 없습니다. 어댑터의 주문 형식이 같아서, 에이전트 코드를 바꾸지 않고 mock에서 Perpl로 옮길 수 있습니다.



컨트랙트가 강제하는 조건

위험 한도

RiskLimits · 11개 필드

- 레버리지
- 고점 대비 손실 (드로다운)
- 주문 사이 최소 블록 수
- 마크 가격 최대 유효시간
- 주문·포지션·전체·블록당 규모
- 변동성 스트레스 테스트 (필드 3개)

거래 조건

TradeTerms · 6개

- 거래 가능한 마켓
- 롱·숏 방향
- 마크 대비 주문 가격 허용 폭
- 하루 거래 횟수
- 하루 손실 한도
- 포지션 최대 보유 시간

수수료

FeeTerms · 2개

- 성과 수수료 (상한 30%)
- 운용 수수료 (상한 연 5%)
- 에이전트에게 현금이 아니라 볼트 지분으로만 지급
- 최고 NAV를 넘은 이익에만 성과 수수료

원칙 주문 전에 막을 수 있는 위반은 그 주문만 취소합니다. 가격이 움직여서 저절로 넘는 위반은 세 단계입니다. 하루 손실은 그날만 일시정지, 가격이 끊기면 정지(재개 가능), 고점 대비 손실·보유 시간 초과는 되돌릴 수 없는 동결.

한도를 넘은 뒤에 일어나는 일

멈춤 3단계

`poke()`

하루 손실: 다음 UTC 날까지 새 위험만 일시정지. 가격 끊김: 정지, 새 가격이 한도 안이면 누구나 `resume()`으로 재개. 고점 대비 손실·보유 시간 초과: 동결, 되돌릴 수 없음.

가격이 유효시간의 3배 넘게 끊기면 정지. 청산은 15분 뒤부터

청산

`unwind()`

누구나 다섯 번에 걸쳐 포지션을 20%씩 단습니다. 포지션을 줄이는 방향으로만 거래하고, 마크에서 1% 넘게 불리한 가격은 받지 않습니다.

운영자가 사라져도 남이 끝까지 달아 줄 보상 구조

출금

`withdraw()`

출금은 동결 중에도 열려 있고, 청산이 현금을 만듭니다. 활성 볼트에서 현금이 모자라면 `requestRedeem()`으로 요청하고, 1일 뒤 누구나 포지션을 줄여 현금을 마련합니다.

취소는 `cancelRedeem()`. 피드가 멈추면 `withdrawUnpriced()`로 현금 뭉 출금

정직하게 손실 한도는 “넘는 순간 멈춤”이 아닙니다. 가격이 한 번에 크게 움직이면 동결 시점 손실이 한도를 넘습니다 (10-06 테스트넷 데모: 한도 3%, 동결 시 4.24%). 보장하는 것은 위반 뒤에 새 위험이 더 쌓이지 않는다는 것입니다.

가격 유효시간 조건은 체인이 지킬 수 있어야 합니다

예: `maxMarkAgeSeconds = 4초`인 볼트는 4초보다 오래된 가격으로 주문을 받지 않습니다. 가격은 트랜잭션으로 갱신되므로, 블록 간격이 이 조건의 바닥이 됩니다.

12초 블록 체인

가격 갱신 12초마다
각 구간의 2/3는 `MarkTooOld`



블록이 짧은 체인

가격 갱신 1초마다
항상 통과



■ 주문 가능

■ 가격이 낮아 취소

로컬 빌드에서 오라클을 12초로 묶는 스위치로 재현 · 36초 구간

블록 간격은 바닥일 뿐이고, 가격도 그만큼 자주 넣어야 합니다. 테스트넷 데모는 오라클이 5초마다 갱신해서 가장 엄격한 볼트가 10초 조건을 씁니다. Monad 위에 만드는 더 직접적인 이유는 실제 거래소 `Perpl`이 Monad에 있기 때문입니다. `Perpl`의 마크는 측정 시 1~50초 전 값이었고, `Perpl` 자신도 60초 넘는 가격은 거부합니다.

실제 거래소 연결: Perpl 어댑터

가격을 운영자가 정하지 않음

Perpl이 체인에 쓰는 마크와 그 타임스탬프를 그대로 씁니다. 가격 유효시간 검사가 실제 거래소의 시계로 돌아갑니다.

볼트마다 Perpl 계정 하나

마진은 거래하는 순간에만 볼트에서 Perpl로 가고, 남는 돈은 바로 돌아옵니다. 출금 대상은 그 볼트뿐입니다.

전부 체결 아니면 취소

주문은 지정가에 즉시·전량 체결로만 냅니다. 거래소가 보고한 포지션이 예상과 다르면 트랜잭션 전체를 되돌립니다.

에이전트 코드는 그대로

mock 거래소와 같은 주문 형식이라 에이전트를 바꿀 필요가 없습니다.

Monad 테스트넷 포크 테스트

1. 누구나 등록 (createMandate)
2. 500 aUSD 입금
3. 0.001 BTC 롱, Perpl에 기록
4. 한도 넘는 주문은 Perpl 전에 거부
5. 롱에서 숏으로 전환
6. 보유 시간 초과로 동결
7. 청산 끝까지, Perpl 계정 비용
8. 배분자 전액 출금

10회 연속 통과 (2026-10-06)
같은 날 테스트넷 배포, Perpl에서 체결 10건,
한도 초과 주문 5건 온체인 거부

데모 영상

The screenshot shows the MANDATE web application interface. At the top, there is a navigation bar with the MANDATE logo and several menu items: Market (highlighted), Agent, Allocate, Batch, Privacy, Launch, and Live Risk. A dropdown menu for 'Monad testnet · chain 10143' is open, showing a 'Connect' button. Below the navigation bar, the main content area has a dark blue background with the text 'AUTONOMOUS CAPITAL · ONCHAIN RISK' and a large headline: 'Back the mandate. Not the track record.' Below this, a paragraph explains: 'Agents execute; they never hold. Withdrawal stays with the allocator and every order clears an onchain RiskGuard first. Every number below is read from contract state on this chain.'

On the right side, there is a summary of market data:

- MARKET AUM**: \$31,000 (5 vaults, one venue)
- LIVE MANDATES**: 05 (0 frozen by RiskGuard)
- ETH MARK**: \$2001.60 (block #68600706 · oracle every 15s)

At the bottom left, there is a section titled 'LIVE MARKET' and 'Mandate book'. At the bottom right, there is a legend: ● Active ● Over limit ● Frozen ● Closed.

영상 순서

1. Launch로 새 에이전트 등록
2. 조건표 확인
3. 1,000 USDC 입금
4. 한도 안 주문 통과, 넘는 주문 거부
5. -2% 충격, poke로 동결
6. unwind 청산, 출금
7. 탐색기에서 트랜잭션 확인

2분 59초 · 테스트넷 공개 데모
거래소와 USDC는 mock

만든 것과 확인한 것

돈

에이전트는 돈을 못 꺼낸다

주문만 낼 수 있고, 맡긴 돈의 출금은 맡긴 사람만. 묶인 묶은 요청 1일 뒤 누구나 줄여 현금화

한도

넘는 주문은 미리 막힌다

거래소에 닿기 전에 컨트랙트가 조건을 검사하고 거부한다

정지

단계별로 멈춘다

하루 손실은 일시정지, 가격 폭락은 정지, 큰 손실은 동결. 포지션은 누구나 나눠 정리

공개

지금 직접 해 볼 수 있다

모나드 테스트넷 공개 데모. 모든 동작이 트랜잭션으로 남는다

어떻게 확인했나

- 코드를 바꿀 때마다 자동 테스트가 돈다: 주문 거부, 동결, 청산, 출금, 등록부터 출금까지 전체 흐름
- 함수를 무작위 순서로 수없이 불러도 "에이전트에게 돈이 넘어가지 않는다", "잠긴 조건은 바뀌지 않는다" 같은 규칙이 깨지지 않는지 검사
- 실제 Perpl 거래소를 복사한 환경에서 등록부터 출금까지 10번 연속 성공
- Monad 테스트넷의 실제 Perpl에서 에이전트 스크립트가 짧게 3회(약 1시간) 거래, 한도 넘는 주문 5번은 체인에서 거부

어디까지 확인했고, 무엇을 보장하지 않나

확인한 것

- Mandate 컨트랙트 전체가 Monad 테스트넷에 배포되어 실제 트랜잭션으로 동작한다
- 실제 거래소 Perpl 테스트넷에 연결해 소액 거래로 확인: 체결 10건, 한도 초과 주문 5건 온체인 거부
- 공개 데모는 누구나 바로 써 볼 수 있도록 테스트용 거래소·USDC와 팀 오라클 가격을 쓴다

설계상 보장하지 않는 것

- 전략의 수익. Mandate는 행동의 범위를 강제하는 인프라다
- 손실이 정확히 한도에서 멈추는 것. 급락하면 넘을 수 있고, 그 뒤 새 위험을 막는다
- 포지션에 묶인 돈의 즉시 출금. 요청 1일 뒤 누구나 현금화를 강제할 수 있다

Mandate

믿음 대신 잠긴 조건으로 맡기는 자본

github.com/jiwon000/mandate